

UNIVERZITET U NOVOM SADU,
FAKULTET TEHNIČKIH NAUKA

Predmet:

Upravljanje razvojem informacionih sistema

Naziv projekta:

Sistem za upravljanje projektima

Profesor: Vladimir Mandić

Asistent: Andrej Katin

Studenti:

Sara Jović IT49-2022
Jovana Zečević IT41-2022
Mila Bikar IT70-2022
Neda Janjičić IT71/2022
Mateja Đokić IT59/2022
Andrej Vukoje IT64/2022

Školska godina 2025/26

Novi Sad, januar 2026. godine

Sadržaj

Opis problema i ciljeva sistema.....	4
Opis korisničkih uloga.....	5
Team member.....	5
Admin.....	7
Project manager.....	7
Client.....	7
Funkcionalni zahtevi.....	8
Autentifikacija i autorizacija.....	8
Upravljanje korisnicima.....	8
Upravljanje projektima.....	8
Sprint menadžment.....	8
Radni paketi i zadaci.....	8
Praćenje utrošenog vremena.....	8
Prilozi / fajlovi.....	9
Fakturisanje i plaćanja.....	9
Notifikacije.....	9
Integracije sa eksternim sistemima.....	9
Nefunkcionalni zahtevi.....	9
Bezbednost.....	9
Skalabilnost.....	9
Održivost.....	10
Performanse.....	10
Arhitektura sistema.....	11
Opšti pregled arhitekture.....	11
Mikroservisna struktura.....	11
Interna struktura servisa.....	11
Opis mikroservisa.....	12
Attachment Service.....	12
Auth Service.....	12
Integration Service.....	13
Notification Service.....	13
Payment Service.....	14
Project Service.....	15
Sprint Service.....	15
Timelog Service.....	16
User Service.....	16
WorkPackage Service.....	17
Opis baze podataka.....	19
Database-per-service pristup.....	19
Migracije i seed podaci.....	19
Entity Framework Core.....	19
Opis komunikacije između servisa.....	20
Opis frontend-backend interakcije.....	21

Opis DevOps postavke.....	24
Opis testiranja.....	24
Zaključak.....	26

Opis problema i ciljeva sistema

Sa sve bržim razvojem organizacija raste i potreba za sistemima koji mogu da isprate sve složenije poslovne zahteve. Organizacije često realizuju više projekata istovremeno, što zahteva visok nivo koordinacije, planiranja i kontrole. Neophodno je imati jasan uvid u to koji zaposleni rade na određenom projektu, koji su zadaci definisani i koji su rokovi za njihovu realizaciju. Ručno vođenje ovih informacija, bez centralizovanog sistema, otežava praćenje napretka projekata, kao i efikasnu raspodelu odgovornosti među članovima tima. Nedefinisane uloge i nejasna raspodela zadataka mogu dovesti do konflikata, grešaka u radu i zastoja u realizaciji projekta. U takvim okolnostima, pojedini članovi tima mogu ostati nedovoljno angažovani, dok sa druge strane može doći do preopterećenja određenih zaposlenih. Nedostatak adekvatne organizacije često dovodi do probijanja rokova i povećanja troškova, kao i narušavanja međuljudskih odnosa unutar timova. Takva radna atmosfera negativno utiče na produktivnost zaposlenih i kvalitet realizacije projekata, što dodatno naglašava potrebu za razvojem sistema za upravljanje projektima.

Glavni cilj jeste kreiranje centralizovanog sistema koji omogućava praćenje svih neophodnih aspekata za realizaciju jednog projekta na jednom mestu. Potrebno je omogućiti efikasno planiranje projekta i njegovo razlaganje na pojedinačne zadatke i njihove rokove. Neophodno je obezbediti jasan pregled svih funkcionalnosti i njihovo jednostavno korišćenje u cilju brze i efikasne organizacije. Sistem treba da omogući jasnu raspodelu uloga i odgovornosti među članovima projektnog tima, kao i praćenje njihovog angažovanja na pojedinačnim zadacima, što bi trebalo da rezultira boljom koordinacijom rada i smanjivanju mogućih nesporazuma i konflikata između zaposlenih. Takođe, jedan od važnih ciljeva jeste obezbeđivanje kontinuiranog uvida u status projekta u realnom vremenu, što omogućava pravovremeno prepoznavanje potencijalnih problema i kašnjenja.

Opis korisničkih uloga

Team member

1. As a team member, I want to be aware of dependencies that affect my tasks, so that I can avoid idle time.

Acceptance Criteria:

- Korisnik na listi svojih zadataka vidi jasnu oznaku (ikonu) ukoliko je zadatak blokiran od strane drugog zadatka.
- Klikom na oznaku zavisnosti, korisnik dobija informacije o blokirajućem zadatku: njegov naziv, trenutni status i osobu koja je za njega zadužena.
- Sistem onemogućava promenu statusa zadatka u "In Progress" ukoliko svi zadaci od kojih on zavisi nisu prethodno prebačeni u status "Done".
- Korisnik može samostalno da doda novu zavisnost između zadataka na kojima radi.

Procena složenosti: 8 Story Points.

Obrazloženje: Zahteva složenu proveru stanja (state check) unutar Work Management servisa i integraciju sa relacionom tabelom zavisnosti (Dependency). Takođe, potrebna je logika na frontendu koja vizuelno ograničava akcije korisnika na osnovu statusa blokirajućih taskova.

2. As a team member *I want* to see how many different projects I am actively involved in at the same time *so that* I can better manage my focus and avoid excessive context switching.

Acceptance Criteria:

- Korisnik može da pristupi pregledu svojih aktivnih projekata
- Prikazuje se broj projekata na kojima je korisnik aktivan član
- Za svaki projekat prikazuje se naziv i uloga korisnika
- Lista je sortirana
- Klikom na projekat otvara se stranica projekta

Procena složenosti: 8 Story Points

Obrazloženje: Mikroservisna arhitektura zahteva komunikaciju između više servisa (User, Project, Task), implementaciju API gateway-a za agregaciju podataka, sinhronizaciju između servisa, složenije error handling i integration testove.

3. As a team member I want to view a list of all team members, so that I know who is working on the project and can communicate with them.

Acceptance Criteria:

- Korisnik pristupi listi članova sa stranice projekta
- Prikazuju se svi aktivni članovi projekta
- Za svakog člana prikazuje se ime, prezime i uloga
- Lista podržava pretragu po imenu
- Klikom na člana otvara se profil

Procena složenosti: 5 Story Points

Obrazloženje: Potrebna komunikacija između Project i User servisa, API pozivi za dobijanje User detalja, error handling za nedostupnost servisa.

4. As a team member I want to be notified when someone mentions me in a comment or discussion, so that I can quickly respond to messages relevant to me.

Acceptance Criteria:

- Sistem prepoznaje @ simbol i korisničko ime unutar polja za komentare.
- Generiše se obaveštenje u realnom vremenu unutar aplikacije.
- Klik na notifikaciju vodi korisnika direktno na lokaciju spominjanja.
- Spomenuto ime u tekstu je vizuelno istaknuto (npr. drugom bojom).

Procena složenosti: 8 Story Points

Obrazloženje: Potrebna je komunikacija između Notification servisa, User servisa i WorkPackage servisa (u kojem se nalaze komentari). Implementacija obuhvata asinhroni prenos poruka između servisa, logiku za parsiranje teksta radi identifikacije korisnika i razvoj sistema za distribuciju obaveštenja u realnom vremenu.

5. As a team member I want to be able to comment on tasks to clarify my work or given requirements.

Acceptance Criteria:

- Korisnik može da doda komentar na entitet Task.
- Svaki komentar prikazuje ime autora i vreme objave.
- Omogućeno je uređivanje i brisanje sopstvenih komentara.
- Komentari su hronološki prikazani unutar zadatka.

Procena složenosti: 5 Story Points

Obrazloženje: Potrebna je implementacija CRUD operacija za entitet Comment koji je povezan sa Task entitetom unutar istog servisa. Neophodno je obezbediti povezivanje sa ProjectMember entitetom radi validacije prava pristupa i identifikacije autora, kao i definisanje API endpoint-a za komunikaciju sa frontend-om.

6. As a team member, I want to attach files or documentation to the project, so that relevant resources are available to everyone working on the project.

Acceptance Criteria: Team member može da uploaduje fajl na projekat, podržani su standardni formati (PDF, DOCX, XLSX, PNG, JPG), svi članovi projekta mogu da vide i preuzmu dodate fajlove, sistem jasno prikazuje naziv fajla, autora i datum dodavanja.

Procena složenosti: 5 Story Points

7. As a team member, I want to update the status of my tasks, so that progress is accurately tracked.

Acceptance Criteria: Team member može da promeni status samo zadataka koji su mu dodeljeni, statusi su jasno definisani i unapred konfigurisani, promena statusa je vidljiva svim članovima tima, sistem beleži poslednju promenu statusa (vreme i korisnik).

Procena složenosti: 3 Story Points

Admin

1. As an admin I want to deactivate user accounts so that inactive users no longer have access to the system.
2. As an admin, I want to assign a user's role so that users have correct permissions.
3. As an admin, I want to view all projects so that I can monitor overall system usage.
4. As an admin, I want to assign a project manager to any project so that every project has a responsible person.
5. As an admin, I want an overview of all payments in the system so that I can track the financial flow of the organization.

Project manager

1. As a project manager I want to assign tasks to team members so that responsibilities within the project are clearly defined.
2. As a project manager, I want to add, remove, or reassign tasks during the project, so that the project stays flexible and aligned with changing requirements.
3. As a project manager, I want to see a project dashboard so that I can get an overview of project health and progress.
4. As a project manager, I want to set and adjust deadlines and task priorities, so that the team focuses on the most important work and meets project goals.
5. As a project manager, I want to issue an invoice for a project with individual items so that the client sees exactly what is being charged.

Client

1. As a client I want to see a confidence level of project delivery, not just percentage complete so *that* I can understand the real likelihood of meeting agreed deadlines.
2. As a client I want to see recent project updates so that I stay informed without needing direct communication with the team.
3. As a client I want to be able to see all current tasks and their progress so I know how much work is left.
4. As a client, I want to see all invoices issued for my project so that I have a clear overview of what I am being charged.

Funkcionalni zahtevi

Funkcionalni zahtevi opisuju specifične mogućnosti i ponašanja aplikacije; sve akcije koje korisnik ili drugi sistemi mogu da izvrše nad sistemom.

Autentifikacija i autorizacija

- Sistem obezbeđuje siguran pristup kroz login mehanizam zasnovan na email-u ili korisničkom imenu i lozinki, uz automatizovano osvežavanje i invalidaciju bezbednosnih tokena pri odjavi. Pored toga, omogućen je pregled svih aktivnih sesija korisnika, kao i administrativna kontrola koja dozvoljava trenutno poništavanje svih aktivnih sesija sa jednog mesta.

Upravljanje korisnicima

- Ovaj modul pokriva kompletan životni ciklus korisničkih naloga, uključujući registraciju, izmenu, brisanje, kao i aktivaciju ili deaktivaciju korisnika. Administratore podržava kroz napredno filtriranje po ulogama i aktivnostima, dinamičku promenu rola, validaciju kredencijala i detaljan *audit log* koji bilježi sve ključne akcije korisnika u sistemu.

Upravljanje projektima

- Omogućeno je kompletno kreiranje, pregled, izmena i brisanje projekata uz filtriranje prema statusu ili dodeljenim korisnicima. Sistem takođe pokriva organizaciju tima kroz upravljanje članovima projekta, praćenje ključnih prekretnica (*milestones*) i vođenje svih pratećih projektnih zahteva.

Sprint menadžment

- Podržava agilne metodologije kroz punu kontrolu nad sprintovima u okviru pojedinačnih projekata. Korisnici mogu kreirati, menjati i brisati sprintove, kao i pratiti njihov tok kroz definisane statusne: *NotStarted*, *Active*, *Completed* i *Cancelled*.

Radni paketi i zadaci

- Operativni rad se organizuje kroz radne pakete i zadatke sa podrškom za podzadatke, međusobne zavisnosti i vođenje projektnog *backlog-a*. Funkcionalnost obuhvata promenu statusa zadataka (*ToDo*, *InProgress*, *InReview*, *Done*, *Blocked*), preraspodelu drugim članovima tima, premeštanje između radnih paketa i diskusiju putem komentara.

Praćenje utrošenog vremena

- Sistem omogućava precizno beleženje, izmenu i brisanje provedenog vremena na konkretnim zadacima. Omogućen je jasan uvid i izveštavanje o utrošenim satima, kako na nivou pojedinačnog zadatka, tako i na nivou celog projekta.

Prilozi / fajlovi

- Pruža kompletnu podršku za upravljanje dokumentacijom vezanom za zadatke i projekte. Korisnici mogu bezbedno otpremati i preuzimati priloge, pregledati njihove detalje, menjati pripadajuće metapodatke i uklanjati nepotrebne fajlove.

Fakturisanje i plaćanja

- Pokriva finansijski segment kroz izdavanje faktura vezanih za konkretan projekat, sa pojedinačnim stavkama i praćenjem statusa naplate (Unpaid, Paid, Cancelled). Ukupan iznos fakture sistem računa iz stavki, plaćena faktura se zaključava za izmene, a podržane su i delimične uplate uz automatsko praćenje preostalog duga. Vodi se evidencija svih uplata po korisniku ili fakturi, sa statusima obrade (Pending, Completed, Failed, Refunded). Pristup podacima ograničen je na projekte na kojima je korisnik član.

Notifikacije

- Sistem obezbeđuje pravovremeno informisanje korisnika kroz automatsko kreiranje i slanje obaveštenja. Korisnici imaju centralizovani pregled svih pristiglih notifikacija uz mogućnost njihovog označavanja kao pročitanih.

Integracije sa eksternim sistemima

- Omogućava bezbedno povezivanje i upravljanje eksternim servisima i API ključevima. Radi maksimalne bezbednosti, svi osetljivi podaci i akreditivi pri skladištenju automatski se šifruju primenom *DataProtection* mehanizama.

Nefunkcionalni zahtevi

Nefunkcionalni zahtevi definišu kvalitativne karakteristike, ograničenja i standarde sistema.

Bezbednost

Sistem mora zaštititi podatke od neovlašćenog pristupa, izmena i napada primenom enkripcije za podatke u mirovanju i prenosu. Neophodno je implementirati sigurnu autentifikaciju i autorizaciju zasnovanu na ulogama (RBAC), bezbedno upravljanje sesijama i tokenima, zaštitu od standardnih veb ranjivosti.

Skalabilnost

Aplikacija mora biti projektovana tako da efikasno podnosi porast broja korisnika, podataka i saobraćaja bez narušavanja stabilnosti. To podrazumeva mogućnost horizontalnog i vertikalnog proširenja resursa (servera i baza podataka). Mikroservisna arhitektura podržava skaliranje svakog servisa u skladu sa opterećenjem.

Održivost

Kod i arhitektura moraju biti strukturirani tako da omogućavaju lake izmene, nadogradnju i ispravku grešaka uz minimalan rizik od lomljenja postojeće funkcionalnosti. Ovo se postiže pisanjem čistog i dokumentovanog koda, primenom standardizovanih konvencija i koncepata i visokim procentom pokrivenosti automatizovanim testovima (unit/integration).

Performanse

Aplikacija mora obezbediti brzo vreme odziva i visoku efikasnost u obradi zahteva pod definisanim opterećenjem. Konkretno, API odzivi treba da se izvršavaju unutar predviđenih granica (npr. ispod 200 ms za većinu upita), interfejs na frontendu mora omogućiti brzi početni prikaz (FCP) i glatku navigaciju, dok upiti nad bazom i eksternim servisima moraju biti optimizovani radi minimalnog utroška procesorskog vremena i memorije.

Arhitektura sistema

Opšti pregled arhitekture

Sistem za upravljanje projektima projektovan je kao informacioni sistem koji se zasniva na mikroservisnoj arhitekturi. Svaka zasebna funkcionalnost je jedan samostalan servis unutar mikroservisne arhitekture.

Sistem se sastoji od ukupno 9 servisa, gde svaki servis ima svoju nezavisnu bazu podataka i jedan API Gateway servis.

Svi servisi su implementirani kao [ASP.NET](#) CoreWeb API aplikacije, dok je za bazu izabrana MySQL relaciona baza podataka. Komunikacija između servisa realizovana je korišćenjem REST API interfejsa korišćenjem HTTP protokola.

Mikroservisna struktura

Svaki servis je projektovan po principu jedinstvene odgovornosti (*Single Responsibility Principle*) kao potpuno samostalna celina koja upravlja isključivo svojim domenom i pripadajućom, nezavisnom bazom podataka. Ovakva izolacija podataka i logike sprečava tesno sprezanje među komponentama, čime se omogućava da se svaki mikroservis razvija, testira, pokreće i skalira sasvim nezavisno od ostatka sistema.

Interna struktura servisa

Svi servisi su izgrađeni po istom, slojevitom rasporedu foldera.

- **Controllers/** -Sadrži REST API kontrolere za prijem HTTP zahteva i njihovo prosleđivanje nižim slojevima.
- **Data/** -Repository sloj sa parovima interfejs-implementacija za direktan rad sa bazom.
- **Context/** -Smešta EF Core DbContext klase.
- **Migrations/** -Sadrži automatski generisane EF Core migracije za praćenje šeme baze podataka.
- **Models/** -Sadrži domenske modele i objekte za prenos podataka.
 - **DTO/** -Ulazni i izlazni objekti po operacijama.
 - **Enums/** -Enumeracije za statusе i uloge.
- **Profiles/** -Konfiguracije za AutoMapper koji vrši mapiranje između domenskih entiteta i DTO objekata.
- **ServiceCalls/** -Tipizovani HttpClient servisi za komunikaciju sa drugim mikroservisima, podeljeni po ciljnom servisu (npr. Project/, User/).
- **Validation/** -Logika i pravila za validaciju ulaznih podataka.
- **Exceptions/** -Definicije sopstvenih, specifičnih tipova izuzetaka.
- **Properties/** -Sadrži launchSettings.json sa podešavanjima za lokalno pokretanje aplikacije.

Svaki servis pored prethodno navedenih foldera, sadrži i Dockerfile koji služi za konfiguraciju fajlova po okruženjima. Zajedno sa svakim servisom, kreiran je i testni projekat koji strukturalno preslikava glavni servis.

Opis mikroservisa

Attachment Service

Osnovna uloga i protok uvoza:

Servis upravlja celokupnim životnim ciklusom fajlova vezanih za zadatke - od kreiranja metapodataka, preko otpremanja i preuzimanja, do brisanja. Da bi se sprečilo kreiranje "duh" zapisa u bazi za neuspele prenose, primenjuje se dvofazni *upload*: registracija metapodataka (`CreateAttachment`), a zatim potvrda (`ConfirmAttachment`) tek kada fajl bezbedno stigne u MinIO/S3 skladište.

Statusi i ograničenja:

Stanje fajla prati `AttachmentStatus` enum kroz faze: *Uploading*, *Scanning* (provera bezbednosti), *Ready* (spreman za download), *Failed* i *Deleted* (logičko brisanje). Unos se ograničava na nivou DTO-a pomoću `AllowedContentTypesAttribute` (dozvoljeni MIME tipovi: slike, PDF, Office, TXT/CSV, ZIP) i `MaxFileSizeAttribute` (maksimum 25 MB).

Poslovna pravila i izuzeci:

Pristup je strogo definisan i baca specifične izuzetke: otpremanje je dozvoljeno samo određenim ulogama (`RoleCannotUploadAttachmentsException`), izmene vrši samo vlasnik fajla (`NotAttachmentOwnerException`), a pregled zahteva aktivno članstvo na projektu (`UserNotProjectMemberException`) uz obavezan filter po projektu/zadatku (`ProjectContextRequiredException` - jedino Admin ima globalni uvid).

Integracije:

Komunicira sa tri servisa: `ProjectService` (provera projekta i članstva), `WorkPackageService` (validacija postojanja zadatka) i `UserService` (preuzimanje podataka o autoru priloga).

Auth Service

Osnovna uloga i model podataka:

Servis je zadužen za autentifikaciju korisnika i upravljanje njihovim sesijama unutar sistema. Prilikom prijave, servis ne proverava kredencijale samostalno, već poziva `UserService` (`POST api/user/credentials/validate`) radi validacije korisničkog imena i lozinke, čime se čuva princip da svaki servis upravlja isključivo sopstvenim domenom podataka. Nakon uspešne validacije, generiše se par tokena - pristupni JWT token (`AccessToken`) i token za osvežavanje (`RefreshToken`) a podaci o sesiji (`AuthSession`) čuvaju se u bazi radi kasnije kontrole.

Generisanje i struktura tokena:

Generisanje tokena vrši `TokenService`, koji `AccessToken` kreira sa claim-ovima korisničkog identifikatora (`sub`), imena i uloge (`Role`), potpisanim protiv zajedničkog tajnog ključa (`Jwt:Key`), sa definisanim izdavaocem (`Issuer=AuthService`) i publikom (`Audience=UrisApi`). `RefreshToken` se generiše kriptografski bezbedno (`RandomNumberGenerator`, 64 bajta) i ne nosi nikakve podatke o korisniku - služi isključivo kao neprozirni ključ za pronalaženje odgovarajuće sesije u bazi. Trajanje oba tokena je konfigurabilno (`AccessTokenExpirationMinutes`, `RefreshTokenExpirationDays`).

Poslovna pravila i upravljanje sesijama:

Servis omogućava osvežavanje isteklog pristupnog tokena putem važećeg RefreshToken-a, bez ponovnog unosa kredencijala, kao i odjavu korisnika (logout) koja opoziva samo tekuću sesiju. Poseban endpoint (revoke/{userId}) omogućava trenutno poništavanje svih aktivnih sesija određenog korisnika odjednom - ovu funkcionalnost interno poziva UserService prilikom promene korisničke uloge, kako bi izmena prava pristupa odmah stupila na snagu umesto da čeka istek postojećeg tokena. Administrator dodatno ima uvid u sve aktivne sesije korisnika (sessions/{userId}).

Integracije:

Servis komunicira isključivo sa UserService-om, sinhrono prilikom prijave (validacija kredencijala); obrnuta smer komunikacije (UserService - AuthService, prilikom promene uloge) inicira UserService, ne AuthService.

Integration Service

Osnovna uloga i model podataka:

Servis omogućava evidenciju i upravljanje integracijama sa eksternim sistemima. Entitet Integration čuva tip integracije (Type), status aktivnosti (Status) i API ključ, koji se nikada ne čuva u čitljivom obliku već isključivo enkriptovan (ApiKeyEncrypted).

Bezbednost API ključeva:

Enkripcija/dekripcija realizovana je kroz ASP.NET Core Data Protection API (DataProtectionApiKeyProtector) sa reduciranim "purpose" stringom radi izolacije od ostalih upotreba u istoj aplikaciji, a ključevi za enkripciju perzistiraju se na fajl-sistemu kako bi ostali validni i posle restarta servisa. Pri svakom citanju integracije (GetAll, GetById) servis vraća samo maskiranu verziju ključa (ApiKeyMasked - vidljiva su poslednja najviše 4 karaktera), nikad ceo plain-text ključ.

Poslovna pravila:

Prilikom kreiranja integracija se automatski aktivira (Status = true). Ažuriranje (PUT) menja tip i status integracije, dok se API ključ rotira (ponovo enkriptuje) samo ako je nova vrednost eksplicitno prosledjena u telu zahteva.

Integracije:

Servis je trenutno samostalan i ne komunicira sinhrono sa ostalim mikroservisima.

Notification Service

Osnovna uloga i model podataka:

Servis se stara o kreiranju i distribuciji notifikacija korisnicima sistema. Entitet Notification čuva primaoca (UserId), tekst poruke (Description), tip notifikacije (Type) i status procitanosti (IsRead), uz vremenski zig kreiranja (CreatedAt).

Poslovna pravila:

Notifikacija se pri kreiranju uvek postavlja kao neprocitana (IsRead = false), a jedina dozvoljena izmena nad postojećom notifikacijom jeste njeno označavanje kao procitane

(PUT /notifications/{notificationId}) - notifikacije se na drugi način ne azuriraju niti brišu. Pregled notifikacija je uvek filtriran po korisniku (parametar userId je obavezan) i sortiran od najnovije ka najstarijoj.

Integracije:

Notification Service ne poziva druge servise - deluje isključivo kao prijemnik: WorkPackageService, UserService i drugi servisi ga pozivaju (POST /notifications) kada nastupi događaj relevantan za korisnika (npr. otključavanje blokiranog zadatka, promena uloge, pominjanje u komentaru), prosledjujući ugovoreni format tela { userId, message, type }.

Payment Service

Osnovna uloga i model podataka:

Servis pokriva finansijsku stranu projekta, odnosno fakturisanje odrađenog posla klijentu. Obuhvata tri entiteta: Invoice kao root agregata (pripada tačno jednom projektu preko ProjectId, nosi datum izdavanja, ukupan iznos, status i izdavaoca), InvoiceItem kao pojedinačnu stavku fakture (opis, jedinična cena, količina i izračunat iznos) i Payment kao evidenciju uplate po fakturi (iznos, datum, status i platilac). Svi novčani iznosi čuvaju se kao decimal(18,2), a ne kao double, jer double zbog binarne reprezentacije zaokružuje vrednosti pa se zbir stavki razilazi sa ukupnim iznosom fakture.

Poslovna pravila:

Ukupan iznos fakture nikada se ne unosi ručno, već ga servis računa kao zbir svojih stavki, pri čemu se iznos stavke dobija množenjem jedinične cene i količine. Preračunavanje se izvršava pri svakoj izmeni stavki, čime se sprečava da se iznos na fakturi razide sa stavkama koje je čine. Faktura mora imati najmanje jednu stavku da bi bila izdata, a datum izdavanja ne sme biti u budućnosti, što obezbeđuje NotFutureDateAttribute.

Plaćena faktura zaključava se za sve izmene, uključujući dodavanje, izmenu i brisanje stavki, kao i brisanje same fakture. Uplata ne sme premašiti preostali dug, koji se računa kao razlika ukupnog iznosa i zbira već izvršenih uspešnih uplata, pa su delimične uplate podržane. Status fakture prati uplate automatski: prelazi u Paid kada zbir uspešnih uplata dostigne ukupan iznos, a brisanjem uplate vraća se u Unpaid. Brisanjem fakture kaskadno se brišu i njene stavke i uplate.

Stanje fakture prati InvoiceStatus enum (Unpaid, Paid, Cancelled), a stanje uplate PaymentStatus enum (Pending, Completed, Failed, Refunded). Za razliku od izuzetaka, poslovna pravila se signaliziraju kroz OperationResult sa enumeracijom ishoda, jer odbijanje izmene plaćene fakture nije greška u programu nego očekivana situacija koju kontroler prevodi u odgovarajući HTTP status (404, 409 ili 403).

Prava pristupa:

Pristup je uređen kroz JWT autentifikaciju i role-based autorizaciju. Izdavanje, izmena i brisanje faktura i stavki dozvoljeni su ulogama Admin i ProjectManager, evidentiranje uplate dodatno i ulozi Client, dok je brisanje uplate rezervisano isključivo za Admin ulogu, jer briše finansijski trag.

Pored uloge, servis proverava i članstvo na projektu. Fakturu može izdati i platiti samo korisnik koji je evidentiran kao član projekta na koji faktura glasi, a liste faktura i uplata filtriraju se tako da korisnik vidi samo projekte kojih je član, uz ono što je sam izdao ili platio. Admin je izuzet od provere članstva i ima uvid u ceo sistem. Identitet korisnika čita se iz sub claim-a potpisanog tokena, dok X-User-Id header ostaje kao rezerva za slučaj da identitet postavlja API Gateway.

Integracije:

Servis komunicira sa ProjectService-om radi preuzimanja naziva projekta (api/project/{projectId}), provere članstva prilikom izdavanja i plaćanja (api/projectmember/project/{projectId}) i dobavljanja liste projekata kojih je korisnik član pri filtriranju listi (api/project/user/{userId}), kao i sa UserService-om radi prikaza korisničkog imena izdavaoca i platioca (api/user/{userId}). Pošto ProjectService zahteva autentifikaciju, token pozivaoca prosleđuje se dalje kroz AuthForwardingHandler.

Nedostupnost spoljnog servisa ne obara rad. Provera članstva blokira operaciju samo na izričit odgovor da korisnik nije član, dok se pri nedostupnosti servisa operacija propušta, a u prikazu se umesto naziva projekta ili korisnika vraća rezervna vrednost.

Project Service

Osnovna uloga i organizacija projekata:

Servis predstavlja centralni deo sistema za upravljanje projektima - obuhvata kompletan CRUD nad entitetom Project (Name, Budget, Status, Deadline, CreatedAt), kao i organizaciju rada kroz tri povezana entiteta: Milestone (ključna prekretnica sa očekivanim datumom - ExpectedDate), Requirement (projektni zahtev, definisan isključivo tekstualnim opisom - Description) i ProjectMember (upravljanje članstvom korisnika na projektu, uz Status polje koje označava da li je član aktivan ili neaktivan).

Poslovna pravila:

Status projekta prati ProjectStatus enum (Planned, Active, OnHold, Completed, Cancelled). Sistem obezbeđuje da očekivani datum milestone-a (ExpectedDate) ne sme prelaziti rok (Deadline) projekta kome pripada, čime se sprečava logička nekonzistentnost u planiranju - ova provera se izvršava na nivou repository sloja, pre upisa u bazu. Requirement je namerno jednostavan entitet, jer za potrebe projekta zahtev zahteva samo tekstualni opis, bez dodatnih atributa. Sva ulazna polja validirana su kroz DTO validacione attribute (npr. NotEmptyGuid, StringLength).

Prava pristupa:

Pristup je uređen kroz JWT autentifikaciju i role-based autorizaciju. Kreiranje i izmena projekata, milestone-ova i zahteva dozvoljeni su isključivo ulogama Admin i ProjectManager, dok je brisanje projekta rezervisano samo za Admin ulogu. Upravljanje članstvom (dodavanje, izmena, uklanjanje člana projekta) takođe je ograničeno na Admin ulogu, dok operacije čitanja (GET) ostaju dostupne svim autentifikovanim korisnicima - pri čemu korisnici sa ulogom TeamMember i Client vide isključivo projekte na kojima su evidentirani kao članovi, dok Admin i ProjectManager imaju uvid u sve projekte u sistemu.

Integracije:

Servis komunicira sa UserService-om radi provere postojanja korisnika prilikom dodavanja člana na projekat, kao i radi prikaza dodatnih podataka o članovima (korisničko ime, uloga) uz osnovne podatke o članstvu koje čuva u sopstvenoj bazi - čime se izbegava dupliranje korisničkih podataka između servisa.

Sprint Service

Osnovna uloga i validacija datuma:

Zadužen je za organizaciju i praćenje agilnih sprintova u okviru projekta (kreiranje, izmena, brisanje i filtrirani pregledi) definisanih vremenskim okvirom StartDate / EndDate. Pravilnost vremenskog raspona garantuje generički DateGreaterThanAttribute atribut, obezbeđujući da datum završetka uvek bude nakon datuma početka.

Životni ciklus i domenska pravila:

Faza u kojoj se sprint nalazi prati se kroz SprintStatus enum (*NotStarted*, *Active*, *Completed*, *Cancelled*). Za domenska odstupanja koristi se SprintValidationException kao bazna klasa, iz koje nasleđuje i ProjectNotFoundException kada se sprint pokuša vezati za nepostojeći projekat.

Integracije:

Oslanja se isključivo na ProjectService radi provere postojanja projekta, preuzimanja projektnih detalja (potrebnih za rad sa *milestone*-ovima) i dobavljanja liste projekata dodeljenih konkretnom korisniku.

Timelog Service

Osnovna uloga i validacija unosa:

Omogućava članovima tima evidenciju, izmenu, brisanje i pregled utrošenih sati (HoursSpent) po zadatku ili projektu za odabrani datum (Date). Poslovno pravilo je osigurano NoFutureDateAttribute atributom koji strogo zabranjuje unos utrošenog vremena za buduće datume.

Prava pristupa i izuzeci:

Sistem štiti integritet unosa kroz namensko hvatanje grešaka: klijentima je zabranjeno beleženje vremena (ClientCannotLogTimeException), izmene unosa može vršiti samo njegov autor (NotTimelogOwnerException), dok je pristup uslovljen aktivnim članstvom na projektu (UserNotProjectMemberException), uz obavezno postojanje projekta i zadatka.

Integracije:

Ostvaruje komunikaciju sa ProjectService-om (potvrda projekta i članstva), WorkPackageService-om (potvrda zadatka na koji se vreme knjiži) i UserService-om (dobavljanje informacija o korisniku koji je zabeležio rad).

User Service

Osnovna uloga i upravljanje nalogima:

Servis predstavlja centralno mesto za upravljanje korisničkim nalogima u sistemu obuhvata

kompletan CRUD nad entitetom User (ime, korisničko ime, email, kontakt podaci, lozinka, uloga, status aktivnosti), kao i evidenciju istorije korisničkih aktivnosti kroz entitet UserActivityLog (audit log). Uloga korisnika prati UserRole enum (Admin, ProjectManager, TeamMember, Client) koji određuje nivo pristupa u celom sistemu.

Bezbednost lozinki:

Lozinke se nikada ne čuvaju u čitljivom obliku. Prilikom registracije i validacije kredencijala servis koristi PBKDF2 algoritam (Rfc2898DeriveBytes) sa 100.000 iteracija i SHA-256 heš funkcijom, uz jedinstvenu, nasumično generisanu so (salt) od 16 bajtova po korisniku i heš vrednost od 32 bajta. Ovim se obezbeđuje otpornost na napade grubom silom i na napade zasnovane na unapred izračunatim tabelama (rainbow tables).

Poslovna pravila:

Administrator može da filtrira korisnike po pretrazi imena, ulozi i statusu aktivnosti, menja ulogu korisnika i aktivira ili deaktivira nalog. Prilikom promene uloge primenjuje se zaštita od samo-degradacije (self-demotion protection) - administrator ne može sebi oduzeti administratorska prava, čime se sprečava da sistem ostane bez nijednog aktivnog administratora. Svaka promena uloge se evidentira u audit log-u i automatski generiše obaveštenje korisniku.

Integracije:

Servis komunicira sa AuthService-om, kome nakon promene uloge korisnika šalje zahtev za opoziv svih njegovih aktivnih sesija (POST api/auth/revoke/{userId}), kako bi nova prava pristupa odmah stupila na snagu. Takođe komunicira sa NotificationService-om, kome prosleđuje obaveštenje o promeni uloge u ugovorenom formatu ({userId, message, type}). Obrnuto, AuthService poziva UserService radi validacije kredencijala prilikom prijave korisnika.

WorkPackage Service

Osnovna uloga i organizacija rada:

Servis predstavlja centralni deo sistema za upravljanje radnim paketima i zadacima unutar projekta, obuhvata kompletan CRUD nad root entitetom WorkPackage (naziv, opis, status, rok), kao i organizaciju rada kroz četiri povezana entiteta: Task (zadatak sa nazivom, opisom, statusom, prioritetom i zaduženim licem, uz podršku za rekurzivnu hijerarhiju kroz Sub-Task strukturu), Dependency (zavisnost između zadataka - blocker/blocked veza), Comment (komentar na zadatku, egzistencijalno zavisao od Task entiteta) i Backlog (evidencija stavki van trenutnog obima rada, spremnih za planiranje).

Poslovna pravila:

Status radnog paketa prati WorkPackageStatus enum (Planned, InProgress, OnHold, Completed, Cancelled), dok zadatak prati odvojene enumere TaskStatus (ToDo, InProgress, InReview, Done, Blocked) i TaskPriority (Low, Medium, High, Critical). Sistem obezbeđuje da rok (Deadline) radnog paketa ne sme prelaziti rok projekta kome pripada, čime se sprečava logička nekonzistentnost u planiranju (cross-service validacija prema ProjectService). Promenu statusa zadatka sme da izvrši isključivo korisnik kome je zadatak dodeljen (Assigneeld), dok se ostale write operacije nad zadacima, radnim paketima i zavisnostima ograničavaju na uloge ProjectManager i Admin. Komentar može da doda bilo koji

autentifikovan korisnik, dok izmenu i brisanje komentara sme da izvrši isključivo njegov autor. Sva ulazna polja validirana su kroz DTO validacione attribute.

Prava pristupa:

Pristup je uređen kroz JWT autentifikaciju i role-based autorizaciju. Operacije čitanja (GET) dostupne su svim autentifikovanim korisnicima, dok su operacije kreiranja, izmene i brisanja nad WorkPackage, Task, Dependency i Backlog entitetima rezervisane za uloge ProjectManager i Admin. Izuzetak čini promena statusa zadatka, gde autorizacija ne zavisi od uloge već od toga da li je korisnik lice kome je zadatak dodeljen, kao i komentari, gde pravo izmene/brisanja pripada isključivo autoru.

Integracije:

Servis komunicira sa ProjectService-om radi validacije roka radnog paketa u odnosu na rok projekta, kao i provere postojanja projekta prilikom kreiranja radnog paketa. Sa NotificationService-om komunicira radi slanja obaveštenja korisniku kada se blocker zadatak od kog zavisi njegov zadatak završi.

Opis baze podataka

Database-per-service pristup

Ovaj obrazac podrazumeva da svaki mikroservis ima sopstvenu, izolovanu bazu podataka kojoj jedino on može direktno pristupiti. Nijedan drugi servis ne sme direktno da čita niti piše po bazi koja mu ne pripada; sva komunikacija i razmena podataka između servisa odvija se isključivo preko definisanih API endpoint-a ili poruka. Ovim se postiže potpun raspad monolita na nezavisne celine, sprečava se "curenje" domenske logike, a omogućava se i fleksibilnost da svaki servis koristi tip baze koji mu najviše odgovara, kao i potpuno nezavisno skaliranje.

Migracije i seed podaci

Migracije u EF Core-u pružaju mehanizam za kontrolu verzija šeme baze podataka direktno iz C# koda (*Code-First* pristup). Kada se domenski modeli u kodu izmene, EF Core generiše migracione fajlove koji sadrže tačna uputstva kako bazu unaprediti na novu verziju ili vratiti na prethodnu. Na ovaj način, struktura baze se automatski i bezbedno sinhronizuje kroz sve okoline, bez potrebe za ručnim pisanjem SQL skripti.

Entity Framework Core

EF Core je moderni *Object-Relational Mapper* (ORM) za .NET koji služi kao most između objektno-orijentisanog koda i relacione baze podataka. On omogućava programerima da sa bazom komuniciraju koristeći C# klase (domenske modele) i LINQ upite, umesto pisanja sirovih SQL upita. Unutar servisa, EF Core upravlja DbContext-om koji predstavlja sesiju sa bazom i mapira domenske objekte u tabele, vodeći računa o vezama, transakcijama i optimizaciji upita.

Opis komunikacije između servisa

Komunikacija između mikroservisa realizovana je putem sinhronih HTTP poziva korišćenjem tipizovanih klijenata (ServiceCalls sloj), koji se nalaze u okviru svakog servisa koji inicira poziv.

AuthService → UserService

Prilikom prijave korisnika, AuthService poziva UserService da proveri validnost unetih kredencijala pre nego što izda pristupni token.

UserService → AuthService, NotificationService

Kada se korisniku promeni uloga, UserService poziva AuthService da opozove njegove aktivne sesije (da bi se promena uloge odmah odrazila na prava pristupa), i poziva NotificationService da korisnika obavesti o toj promeni.

ProjectService → UserService

Prilikom dodavanja novog člana na projekat, ProjectService poziva UserService da proveri da li korisnik sa datim identifikatorom stvarno postoji, pre nego što se kreira zapis o članstvu. Dodatno, kada se prikazuje lista članova, servis dobavlja korisničko ime i ulogu svakog člana, kako bi se osnovni podaci obogatili čitljivim informacijama za prikaz.

SprintService → ProjectService

Servis proverava da li projekat kome sprint pripada postoji, i dobavlja podatke o roku i milestone-ima projekta. Kada korisnik sa ulogom Client pregleda listu sprintova, SprintService dobavlja listu projekata na kojima je korisnik član, da bi mu prikazao samo relevantne sprintove.

TimelogService → ProjectService, UserService

Prilikom evidentiranja utrošenog vremena, servis proverava postojanje projekta i članstvo korisnika, dobavlja podatke o korisniku, i proverava postojanje zadatka (task-a) na koji se unos odnosi.

WorkPackageService → ProjectService, NotificationService

Prilikom kreiranja ili izmene work package-a, servis dobavlja rok projekta radi poređenja sa rokom work package-a. Odvojeno, šalje notifikaciju kada se task odblokira ili preraspodeli drugom korisniku.

AttachmentService → ProjectService, WorkPackageService, UserService

Prilikom postavljanja ili pregleda priloga, servis proverava postojanje projekta i članstvo korisnika, proverava postojanje zadatka kome se prilog odnosi, i poziva UserService radi administratorskih i vlasničkih provera.

PaymentService → ProjectService, UserService

Pre kreiranja fakture ili evidentiranja uplate, servis proverava da li je korisnik član projekta na koji se to odnosi. Pri prikazu listi dobavlja projekte kojih je korisnik član, kako bi mu se prikazale samo fakture i uplate sa tih projekata, kao i naziv projekta uz podatke o fakturi. Dodatno, dobavlja korisničko ime osobe koja je izdala fakturu ili izvršila uplatu, radi prikaza u potvrdi transakcije. Pošto ProjectService zahteva autentifikaciju, token pozivaoca se prosleđuje dalje.

Opis frontend-backend interakcije

Frontend aplikacija razvijena je kao jedinstvena React (Vite) aplikacija, zajednička za ceo sistem, koja pokriva sve funkcionalne celine opisane kroz devet mikroservisa. Bez obzira na to što je backend podeljen na nezavisne servise sa sopstvenim bazama podataka, korisnik sistem doživljava kao jedinstvenu, koherentnu aplikaciju - tu integraciju obezbeđuje sloj komunikacije koji povezuje frontend sa API Gateway-em.

Opšti princip komunikacije

Frontend nikada ne komunicira direktno sa pojedinačnim mikroservisima, iako svaki od njih (u razvojnom okruženju) ima sopstveni mapirani port. Svaki zahtev sa korisničkog interfejsa upućuje se ka API Gateway-u, koji je jedini servis izložen na standardnom portu (8080) i koji zahtev dalje rutira ka nadležnom mikroservisu (AuthService, UserService, ProjectService, SprintService, WorkPackageService, TimelogService, AttachmentService, PaymentService, IntegrationService, NotificationService). Ovakav pristup obezbeđuje jedinstvenu tačku ulaza u sistem, omogućava da se autentifikacija proverava na jednom mestu pre prosleđivanja zahteva internim servisima, i čini frontend nezavisnim od unutrašnje topologije i eventualnih promena u broju ili rasporedu mikroservisa.

Na frontend strani, sva komunikacija sa API Gateway-em prolazi kroz zajednički HTTP klijent modul, koji standardizuje slanje zahteva, automatsko dodavanje potrebnih zaglavlja, kao i obradu odgovora i grešaka na jednom mestu. Zahvaljujući ovome, komponente ne pozivaju direktno *fetch*, već se oslanjaju na jedinstven, ponovno iskoristiv mehanizam, čime se smanjuje dupliranje koda i olakšava održavanje kada se API ugovor neke funkcionalnosti promeni.

Autentifikacija, autorizacija i upravljanje sesijom

Korisnik pristupa sistemu putem prijave (email/korisničko ime i lozinka), koju frontend prosleđuje ka AuthService-u preko API Gateway-a. Nakon uspešne prijave, korisnik dobija pristupni JWT token koji frontend čuva i automatski prilaže u *Authorization* zaglavlje svakog narednog zahteva, nezavisno od toga kom mikroservisu je taj zahtev na kraju namenjen. Frontend takođe upravlja osvežavanjem tokena kada istekne, kao i njegovom invalidacijom prilikom odjave korisnika.

Token, pored identiteta korisnika, nosi i podatak o njegovoj ulozi (Admin, ProjectManager, TeamMember, Client), na osnovu koje backend servisi sprovode autorizaciju zasnovanu na ulogama (RBAC). Frontend koristi istu informaciju da bi korisnički interfejs prilagodio ulozi korisnika - akcije koje korisnik nema pravo da izvrši (npr. kreiranje projekta, brisanje radnog paketa, dodela uloge) se na interfejsu sakrivaju ili onemogućavaju. Ovo prilagođavanje isključivo je kozmetičke prirode i služi boljem korisničkom iskustvu; svaka autorizaciona provera se nezavisno i obavezno ponavlja na backend strani, tako da frontend nikada nije jedina linija zaštite.

Poseban slučaj autorizacije predstavlja provera vlasništva ili zaduženja (npr. status zadatka može promeniti samo korisnik kome je zadatak dodeljen, komentar može izmeniti ili obrisati samo njegov autor, prilog može izmeniti samo korisnik koji ga je postavio). I ova pravila

frontend delimično anticipira kroz prikaz interfejsa, ali konačna odluka uvek dolazi sa backend strane, u vidu odgovarajućeg HTTP statusnog koda.

Tok tipičnog zahteva

Bez obzira na to kojoj funkcionalnoj celini pripada, interakcija frontend-a i backend-a prati isti obrazac:

1. Korisnička akcija (učitavanje stranice, klik na dugme, potvrda forme) pokreće poziv zajedničkog HTTP klijenta sa odgovarajućom rutom, metodom i, po potrebi, telom zahteva.
2. Zahtev, sa priloženim JWT tokenom, stiže do API Gateway-a, koji ga prosleđuje nadležnom mikroservisu.
3. Mikroservis validira token, proverava autorizaciju, izvršava poslovnu logiku (uključujući, po potrebi, sinhronu pozive ka drugim mikroservisima) i vraća odgovor u JSON formatu, najčešće u vidu DTO objekata prilagođenih potrebama prikaza.
4. Frontend ažurira stanje odgovarajuće komponente na osnovu dobijenog odgovora, bez potrebe za ponovnim učitavanjem cele stranice.
5. Korisnik dobija povratnu informaciju o ishodu akcije (uspeh ili greška) kroz jedinstven sistem notifikacija u interfejsu.

Ovaj obrazac se dosledno primenjuje kroz sve module sistema - od prijave i upravljanja korisnicima, preko projekata, sprintova, radnih paketa i zadataka, do evidencije vremena, priloga, fakturisanja i notifikacija - čime se korisniku obezbeđuje ujednačeno iskustvo bez obzira na to koji mikroservis se u pozadini poziva.

Interakcija po funkcionalnim celinama

Za korisnika u ulozi Admin ili ProjectManager, frontend omogućava upravljanje korisnicima i projektima - kreiranje, izmenu i deaktivaciju naloga, dodelu uloga, kao i kreiranje projekata, definisanje milestone-ova i zahteva, uz odgovarajuće pozive ka UserService-u i ProjectService-u. Prikaz liste članova projekta zahteva da frontend prikaže podatke koje backend prethodno obogati kombinovanjem informacija iz ProjectService-a i UserService-a.

U delu koji se odnosi na planiranje rada, frontend omogućava kreiranje i praćenje sprintova (SprintService), kao i rad sa radnim paketima, zadacima, pod-zadacima, zavisnostima i backlog-om (WorkPackageService). Promena statusa zadatka, dodavanje komentara i uspostavljanje zavisnosti između zadataka odvijaju se kroz iste principe komunikacije, uz vizuelno isticanje blokiranih zadataka na osnovu podataka koje backend vraća o statusu blocker zadatka.

Evidencija utrošenog vremena (TimelogService), upravljanje priložima (AttachmentService) i fakturisanje/plaćanja (PaymentService) prate identičan obrazac - frontend forme i pregledi komuniciraju sa odgovarajućim mikroservisom preko API Gateway-a, uz autorizacione provere zasnovane na članstvu korisnika na projektu i vlasništvu nad konkretnim zapisom.

Notifikacije (NotificationService) predstavljaju poseban vid interakcije: korisnik ih ne generiše direktno, već ih frontend periodično ili pri određenim akcijama preuzima i prikazuje u centralizovanom pregledu, dok se njihovo kreiranje dešava posredno, kao rezultat aktivnosti

u drugim servisima (npr. dodela zadatka, otključavanje blokiranog zadatka, promena uloge korisnika). Klik na notifikaciju u interfejsu vodi korisnika direktno na relevantan deo aplikacije.

Obrada grešaka i korisnička povratna informacija

Odgovori sa greškom (400, 401, 403, 404, 409) obrađuju se centralizovano u zajedničkom HTTP klijentu, nezavisno od toga koji mikroservis ih je vratio, i prevode se u razumljive poruke pre prikaza korisniku - umesto sirove greške servera, korisnik dobija jasno obrazloženje (npr. da ne može da obriše zadatak od kojeg zavise drugi zadaci, da nema dozvolu za akciju, ili da je unet nevažeći datum). Povratna informacija o uspehu ili neuspehu svake akcije prikazuje se kroz jedinstven sistem obaveštenja unutar interfejsa, čime se korisniku obezbeđuje konzistentno iskustvo kroz ceo sistem, bez obzira na to koji mikroservis stoji iza konkretne funkcionalnosti.

Usklađenost sa razvojnim procesom

Kako se sistem razvija po modelu grana-po-funkcionalnost, gde svaki član tima razvija sopstveni mikroservis i pripadajući deo frontend-a u okviru iste grane, promene API ugovora (rute, struktura DTO objekata, statusni kodovi) usklađuju se sa frontend pozivima pre spajanja u granu main kroz pull request i code review. Zajednički HTTP klijent modul i jedinstveni dizajn sistem korišćen u interfejsu obezbeđuju da svi delovi frontend aplikacije, iako ih razvijaju različiti članovi tima za različite mikroservise, komuniciraju sa backend-om na isti način i deluju kao jedinstvena, konzistentna celina.

Opis DevOps postavke

Verzisionisanje koda i orkestracija servisa (Git & Docker Compose)

Sistem se razvija po grana-po-funkcionalnost (feature branch) modelu: svaka nova funkcionalnost ili ispravka se razvija na zasebnoj grani imenovanoj po konvenciji `feature/<naziv-servisa>` ili `fix/<opis-problema>` (npr. `feature/notification-service`, `fix/integration-getall-corrupted-key`), dok se u granu `main` integrise isključivo kroz pull request nakon code review-a od strane ostalih članova tima. Ovakav pristup omogućava paralelan rad na različitim mikroservisima bez međusobnog ometanja i olakšava praćenje istorije promena po funkcionalnosti.

Za lokalno pokretanje citavog sistema koristi se Docker Compose (`docker-compose.yml` u korenu repozitorijuma), koji orkestrira sve mikroservise, zajednicku MySQL instancu (sa odvojenom bazom po servisu) i MinIO objektno skladište za Attachment servis. Svaki servis se gradi iz sopstvenog Dockerfile-a, povezuje na zajednicku `uris-network` mrežu i ceka (`depends_on` sa `healthcheck`-om) da MySQL bude spreman pre pokretanja. API Gateway je jedini servis izložen na standardnom portu (8080) i zavisi od svih ostalih mikroservisa, dok ostali servisi imaju sopstvene mapirane portove za direktan pristup tokom razvoja i testiranja.

Opis testiranja

Testiranje sistema sprovodi se na nivou svakog mikroservisa pojedinačno, kroz zaseban testni projekat koji strukturalno prati organizaciju glavnog servisa (npr. UserService.Tests, AuthService.Tests). Za pisanje i izvršavanje testova koristi se xUnit radni okvir, uz Moq biblioteku za simulaciju (mock) zavisnosti i Microsoft.EntityFrameworkCore.InMemory provajder za testiranje rada sa bazom podataka bez potrebe za stvarnom MySQL instancom.

Testovi su podeljeni u nekoliko nivoa:

- **Unit testovi kontrolera** - testiraju logiku kontrolera izolovano, uz mokovane repozitorijume i servise (Mock<IRepository>, Mock<IExternalService>), čime se proverava ispravno rutiranje zahteva, autorizacija i formiranje HTTP odgovora bez zavisnosti od stvarne baze ili drugih servisa.
- **Unit testovi repozitorijuma** - testiraju stvarnu logiku upita i perzistencije nad EF Core InMemory bazom, gde se za svaki test kreira nova, izolovana instanca baze (Guid.NewGuid() kao naziv), čime se garantuje da testovi ne utiču jedni na druge.
- **Testovi validacije** - proveravaju ispravnost validacionih atributa nad DTO objektima (obavezna polja, granične vrednosti dužine, format email adrese i slično).
- **Integracioni testovi** - kod servisa gde je to primenjeno (npr. Payment, Project, Sprint, Timelog, Attachment), koriste WebApplicationFactory (Microsoft.AspNetCore.Mvc.Testing) kroz IClassFixture, čime se pokreće cela aplikacija u memoriji i testiraju stvarni HTTP pozivi kroz sve slojeve (kontroler → repozitorijum → baza), bez potrebe za pokretanjem servisa kao odvojenog procesa.

Za Auth i User service testni paket obuhvata testiranje kontrolera (UserControllerTests, AuthControllerTests), repozitorijuma nad InMemory bazom (UserRepositoryTests, AuthRepositoryTests), validacije ulaznih podataka (ValidationTests) i, specifično za AuthService, generisanja i validacije JWT tokena (TokenServiceTests). UserService.Tests trenutno sadrži 20, a AuthService.Tests 17 automatizovanih testova, koji se izvršavaju pri svakoj izmeni koda pre spajanja u granu main.

Coverlet.collector paket je uključen u svaki testni projekat radi merenja pokrivenosti koda testovima (code coverage), što omogućava uvid u delove sistema koji nisu adekvatno pokriveni testovima.

Zaključak

Tokom izrade projekta primenjena je mikroservisna arhitektura kroz deset nezavisnih servisa, od kojih je svaki razvijao jedan član tima, sa sopstvenom bazom podataka i jasno definisanim domenom odgovornosti. Ovakav pristup je omogućio paralelan rad na različitim delovima sistema, ali je istovremeno zahtevao dosledno usaglašavanje API ugovora (strukture DTO objekata, statusnih kodova, formata komunikacije) između servisa, kao i između backend i frontend dela aplikacije.

Najveći deo izazova nije proizlazio iz same implementacije pojedinačnih funkcionalnosti, već iz koordinacije rada u timu - usklađivanja konvencija imenovanja, definisanja jasnih pravila autorizacije po ulogama, kao i praćenja izmena koje su posledično zahtevale ažuriranje u više servisa istovremeno. Kroz rad na projektu tim je stekao praktično iskustvo u organizaciji rada preko Git grana po funkcionalnostima, sprovođenju code review procesa i pisanju automatizovanih testova, čime je pokrivenost koda testovima korišćena kao mehanizam za održavanje stabilnosti sistema tokom paralelnog razvoja.

Rad na projektu je zahtevao da se teorijski koncepti primene u praksi - od projektovanja arhitekture, preko implementacije pojedinačnih servisa, do usklađivanja rada u timu na zajedničkom kodu.